



UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE &

TECHNOLOGY

SCHOOL OF ICT

COMPUTER ENGINEERING

Mobile Application Assignment

Flutter Lab 5

Offline Posts Manager

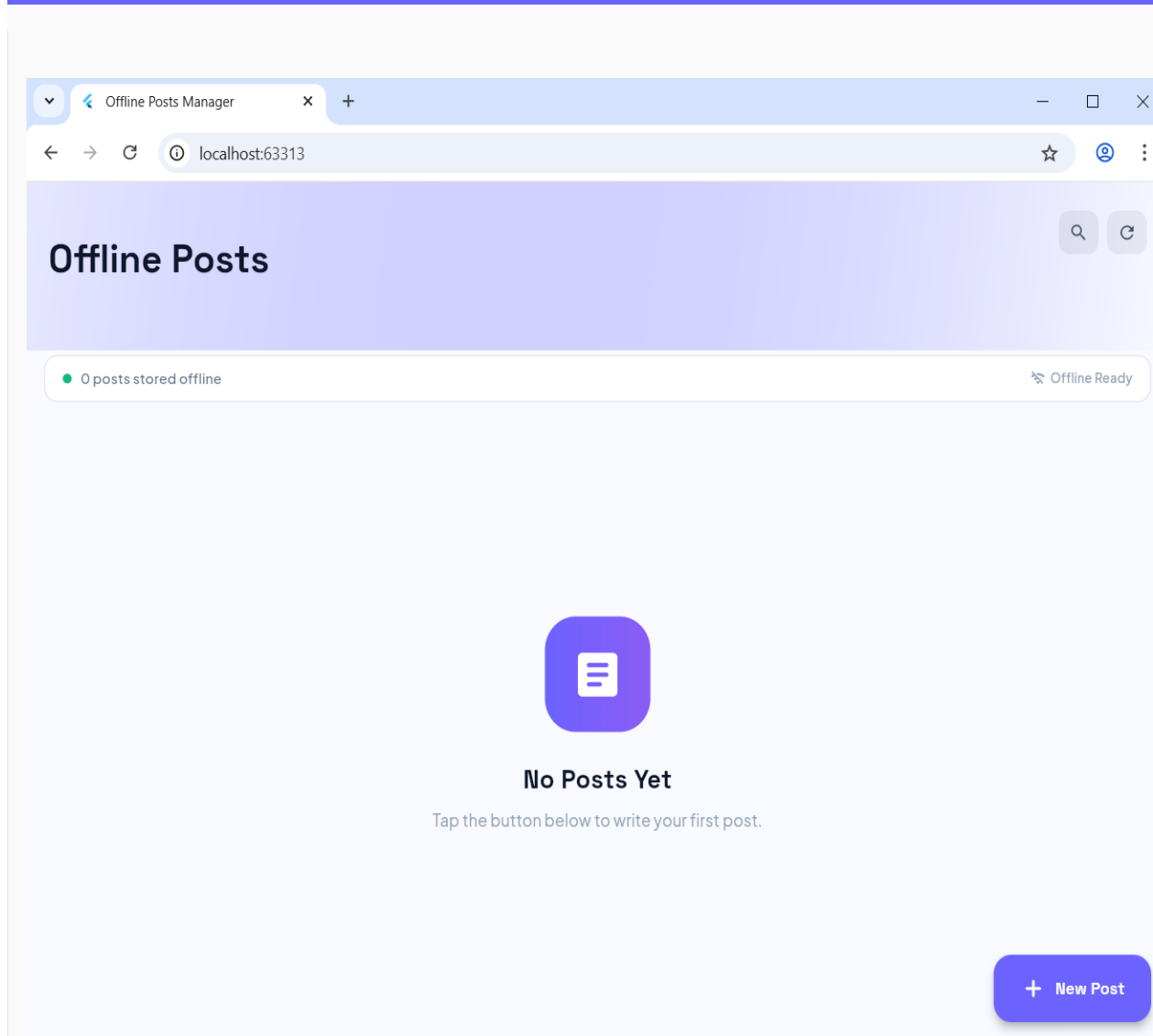
Reg. Number

223021337

Section 1 — Home Screen (Post List)

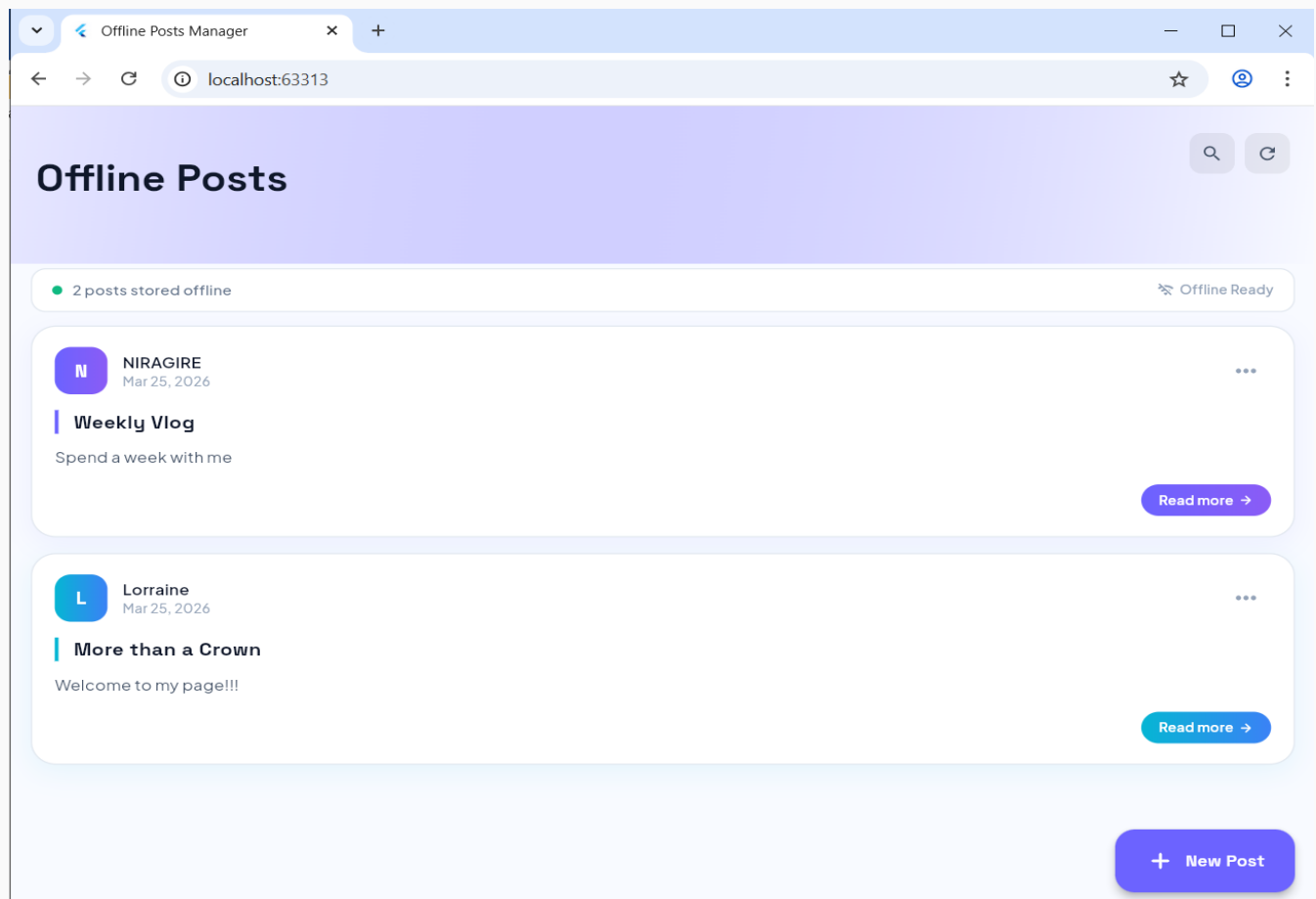
The home screen is the first thing users see. It displays all stored posts, a search bar, and the button to create new posts.

Home Screen — Empty State



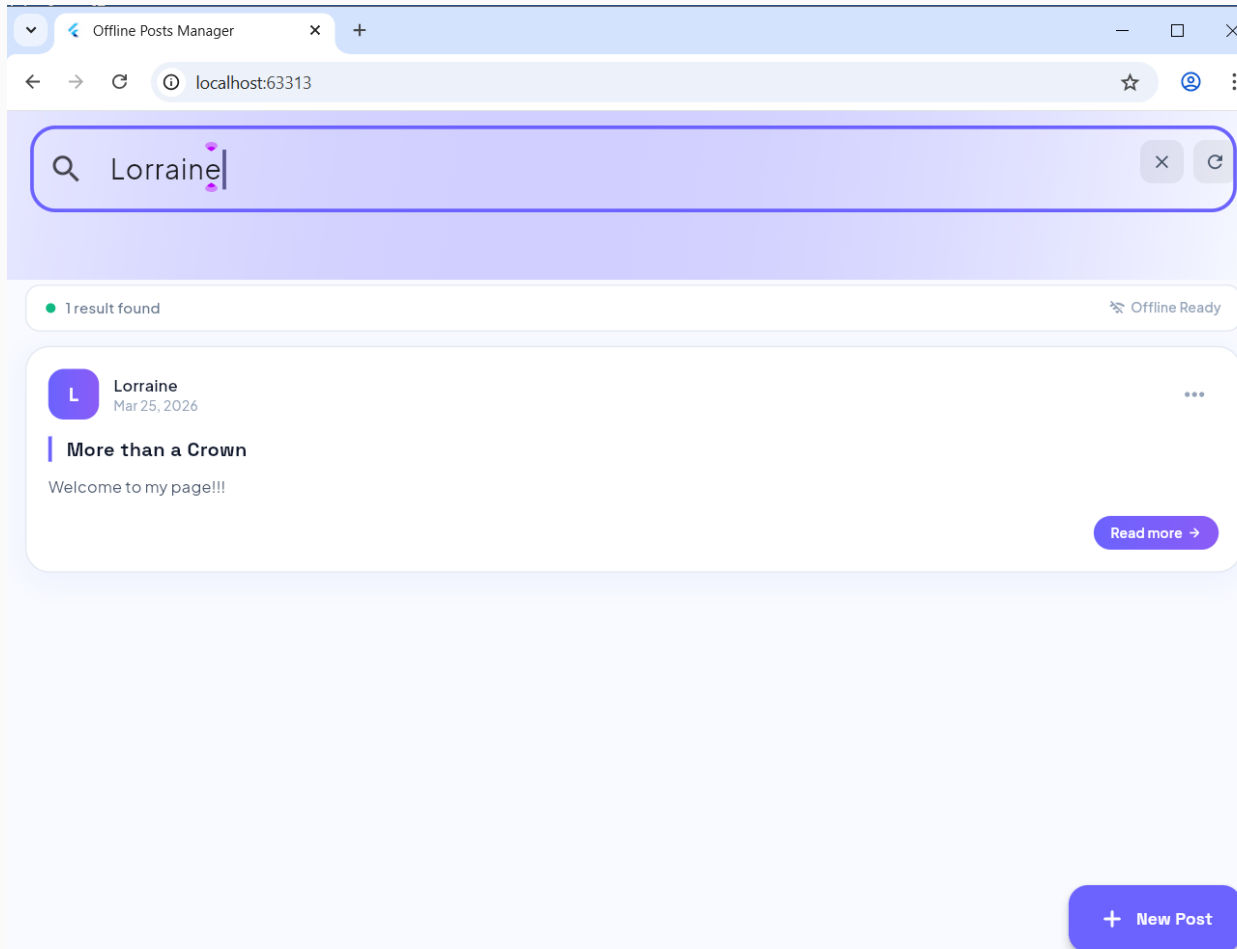
What this shows: The app launched for the first time with no posts yet. Shows the empty state illustration and "No Posts Yet" message with the purple gradient icon.

Home Screen — With Posts



What this shows: The main post list showing multiple post cards. Each card has a colored avatar with initials, the author name, date, title with accent border, content preview, and a gradient "Read more" button.

Home Screen — Search Feature



What this shows: The search field is active and filtering posts in real time. The stats bar updates to show "X results found". Some cards are hidden because they don't match the query.

Section 2 — Create New Post

The form screen allows staff to write and publish new posts. It features a gradient publish button and live form validation.

New Post Form — Empty



New Post

 Author Name

 Post Title

0/120

Post Content



 Publish Post

What this shows: The "New Post" form with all fields empty. Shows the Author Name, Post Title, and Post Content fields with the purple icons and the gradient "Publish Post" button at the bottom.

Validation Error State



New Post

 Author Name

Author required

 Post Title

Title required 0/120

Post Content



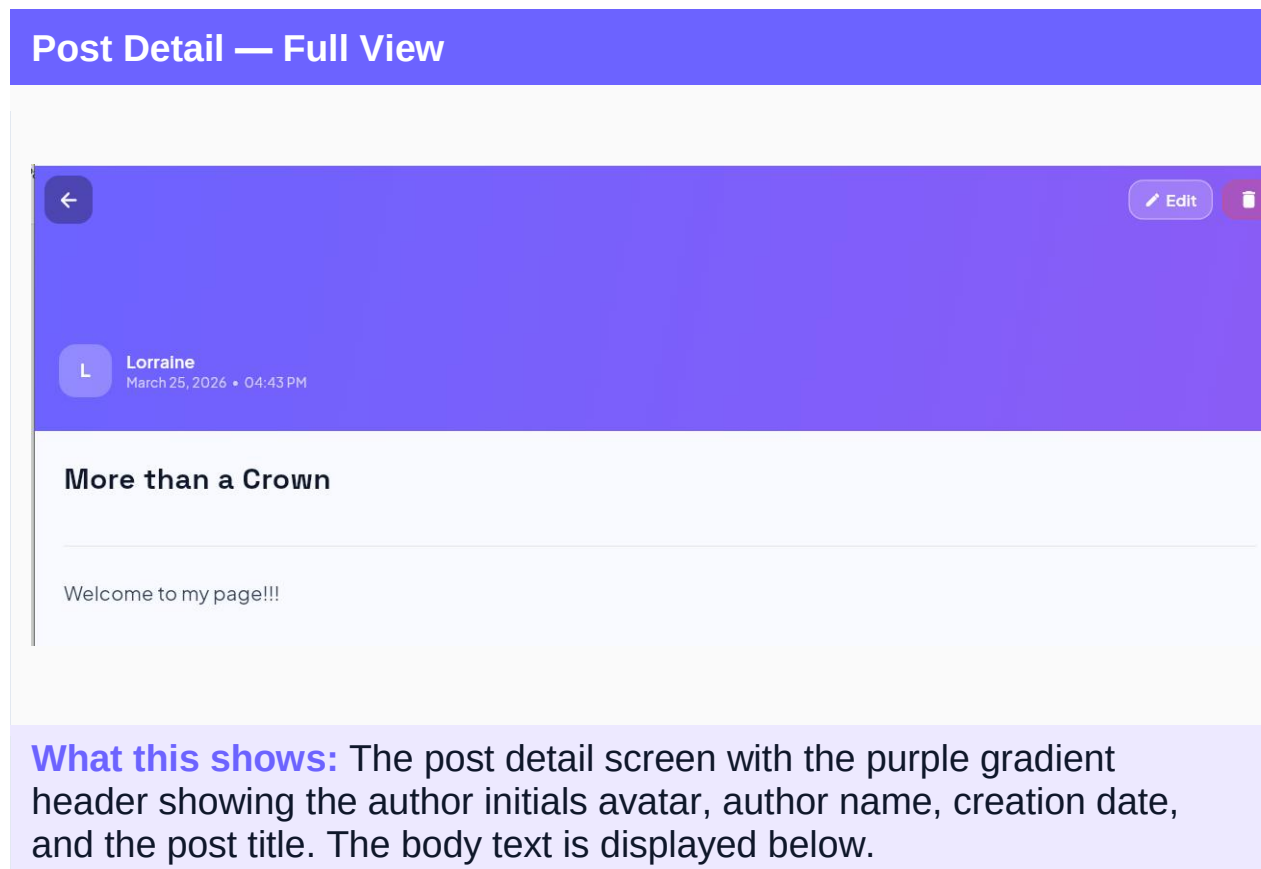
Content required

 Publish Post

What this shows: Form validation triggered — red error messages appear below the fields when the user tries to submit with empty or too-short inputs.

Section 3 — Post Detail View

Tapping any post card opens the detail screen with a gradient header showing the author avatar, title, and full post body.



Section 4 — Edit Existing Post

Users can edit any post by tapping the Edit button on the detail screen or the "... " menu on the card. The form pre-fills with existing data.

Edit Form — Pre-filled

Edit Post

Author Name

Lorraine

Post Title

More than a Crown

Post Content

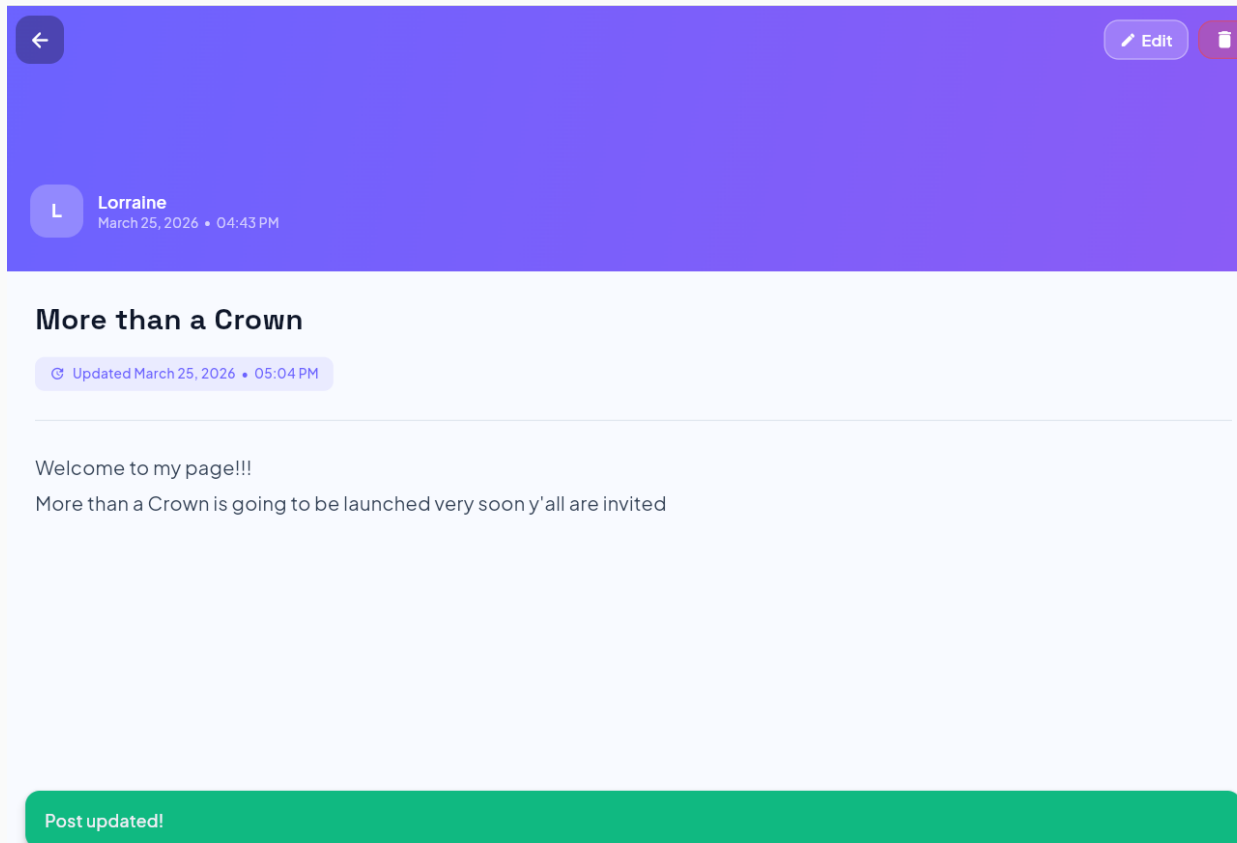
Welcome to my page!!!

17/120

Update Post

What this shows: The same form screen as "New Post" but with "Edit Post" as the title and all fields pre-filled with the existing post data. The button says "Update Post".

Post Updated — Success Snackbar

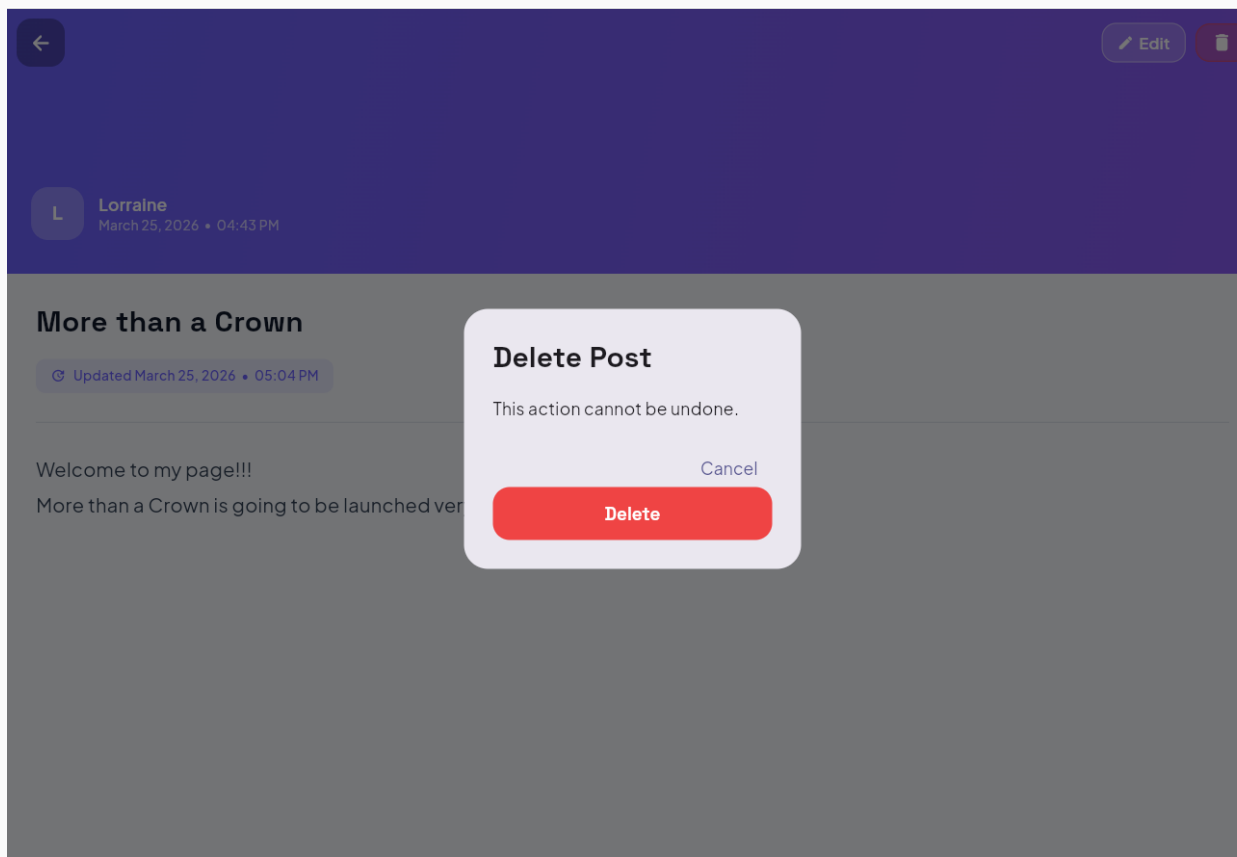


What this shows: After saving, a green "Post updated!" snackbar appears at the bottom of the screen, confirming the operation was successful.

Section 5 — Delete Post

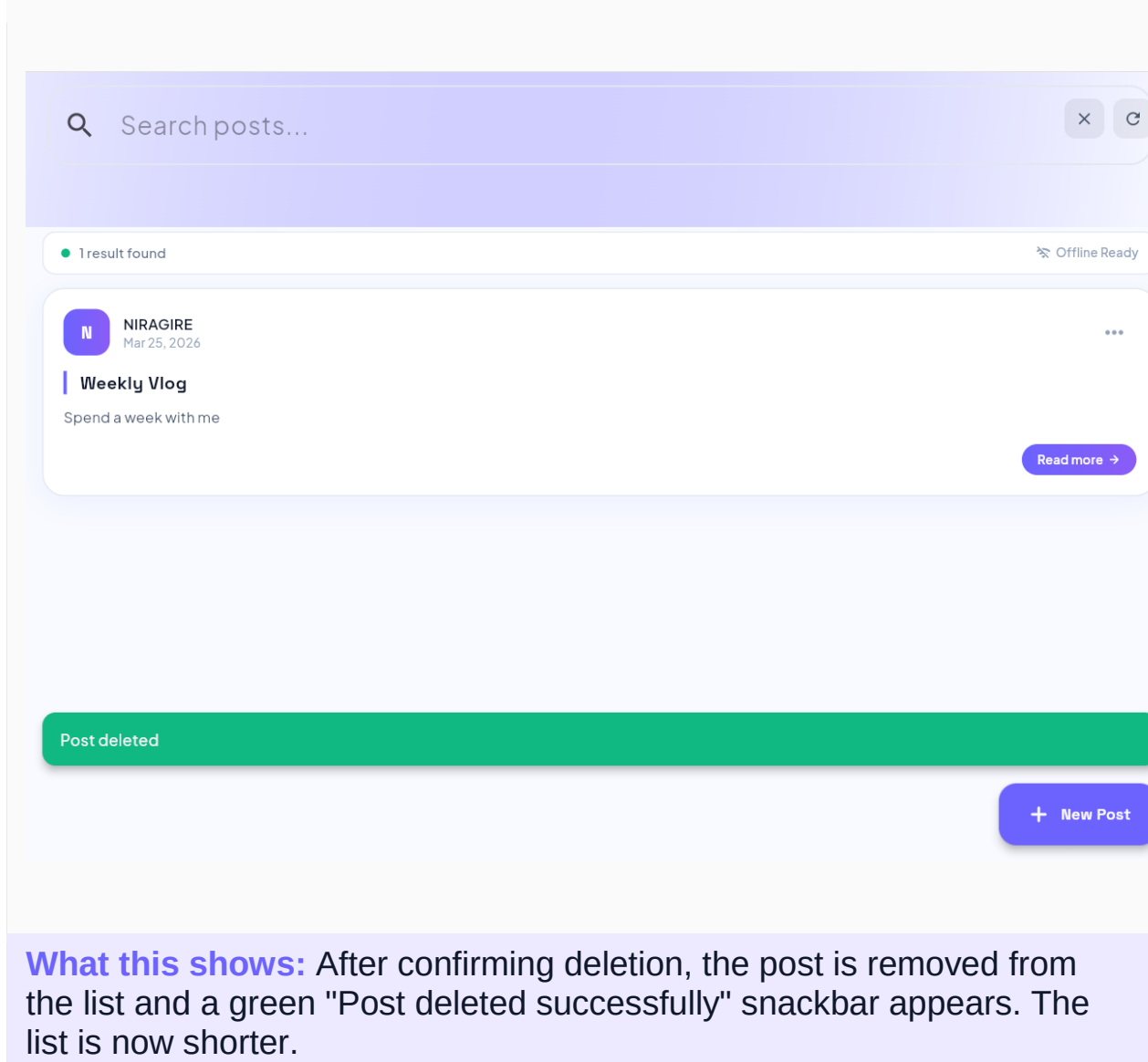
Users can delete posts from the home screen (via the "...\" menu) or from the detail screen. A confirmation dialog protects against accidental deletion.

1.Delete Confirmation Dialog



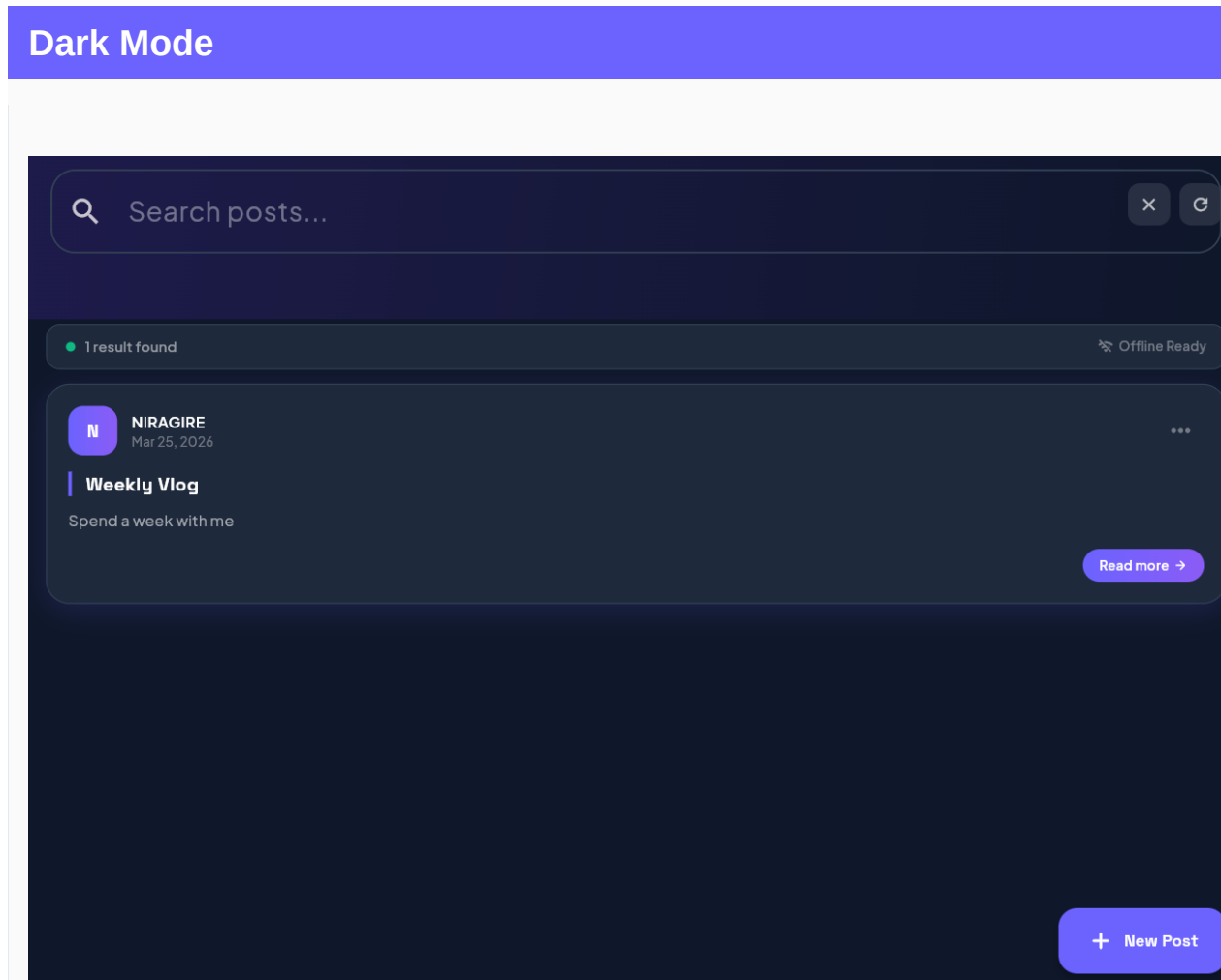
What this shows: The "Delete Post" alert dialog asking "Are you sure you want to delete [title]?" with a Cancel button and a red Delete button.

2. Post Deleted — Success State



Section 6 — Deeper app features

These additional screenshots demonstrate deeper app features and will make your submission stand out.



What this shows: The app running in dark mode. The home screen shows post cards with dark backgrounds, white text, and the same purple gradient accents — proving the theme is fully implemented.

HANDWRITTEN WORK

LAB #5 Flutter - Offline Tests Manager

REG NO: 223021337

Mobile Application Assignments

Q①. Dependencies Used & Why SQLite?

Packages Used

- ⊗. Sqflite : Version ^2.3.0 : Stores data Locally using SQLite database
- ⊗. Path : Version ^1.8.3 : Constructs the correct file-system path to the database file on both Android / iOS
- ⊗. intl : version ^0.19.0 : Formats dates into readable form.

Why SQLite?

- > Works offline (No internet needed)
- > Supports searching, filtering, sorting (SQL Queries)
- > Ensures Data safety (ACID Properties)
- > Data is permanent (Not lost after closing app)
- > Lightweight and fast
- > Easy to use in Flutter with async support

Q②. Database Exception Handling

All Database operations in the app are wrapped in .try-catch blocks,
- Custom DatabaseException for all errors, - Shows error messages instead of crashing

- ⊗. Database Not initialized: → Shows error + Retry option
- ⊗. Insert Fails → Shows error message
- ⊗. UPDATE/DELETE non-existing data → Show 'Post not Found'
- ⊗. Missing ID: Prevent operation before running.
- ⊗. INVALID or Corrupted Data Handling;
 - Form Validation: Title ≥ 5 characters, Body ≥ 10 characters, Author ≥ 2 characters.
 - Prevents bad data before saving.
 - Errors are caught and handled properly.

Q3. SQLite in Flutter - Concepts & Architecture

3.1 Database Structure

- Database → Kthole storage (offline_posts.db)
- Tables → Stores Posts
- Row → One Post
- Column → Fields (id, title, body, Author)

3.2 CRUD Operations:

OPERATION	SQL EQUIVALENT	METHOD IN CODE	SCREEN
CREATE	INSERT INTO posts----	insertPost(Post post)	PostFormScreen
READ	SELECT * FROM posts---	getAllPosts()	PostListScreen
UPDATE	UPDATE posts SET----	updatePost(Post post)	PostFormScreen
DELETE	DELETE FROM posts WHERE id=----	deletePost(int id)	PostListScreen
SEARCH	SELECT * WHERE LIKE?	searchPosts(String q)	PostListScreen

3.3 How Flutter interacts with the Database Asynchronously

Flutter solves this with Dart's `async/await`.

- All SQLite methods return `Future<T>`: a promise that a value will be available later.
- The `await` keyword suspends the current function until the Future resolves, but doesn't block the UI thread.
- The `async` keyword marks a function as one that may contain `await` expressions.
- While data loads, the app shows a CircularProgressIndicator. When the future completes, `setState()` is called to rebuild the widget tree with fresh data.
- The Singleton Database Helper ensures only one database connection is opened. Multiple simultaneous connections to the same SQLite file would cause lock errors.

